

DESCRIPTION D'UNE MISSION | BTS SIO

Prénom – Nom	RAPHEHISON Mandresy
N° mission	E-04

Option	☐ SISR	☒ SLAM
Situation	☐ Formation	☒ Entreprise

Lieu de réalisation	Carrefour CSI – 93 Avenue de Paris, 91300 Massy	
Période de réalisation	Du : 11/03/2026 Au : 13/03/2026	
Modalité de réalisation	☒ VÉCUE ☐ OBSERVÉE	

Intitulé de la mission	Exposer une API pour lister les pipelines de batch
Description du contexte	Dans le cadre de mon alternance à la DSI de Carrefour, une demande interne formalisée via un ticket Jira visait à améliorer la supervision des traitements batch de la plateforme APIM. L'objectif était de centraliser les informations des pipelines en les exposant via une API REST, afin de les rendre exploitables par les outils de monitoring (Grafana, dashboard APIM).

Ressources et outils utilisés	- Poste de développement fourni par l'entreprise - IntelliJ IDEA - Java / Spring Boot - Architecture existante : SourceService, BatchLiveService, ConfService - Accès GitLab (dépôt interne LOGM)
Résultat attendu	- Un endpoint GET <code>/domain/batch/{module}/getAllPipelines</code> retournant pour chaque pipeline : son projet, son type, ses cibles, son débit (throughput) et son état (actif ou en pause) - Format de réponse XML exploitable par les outils de supervision
Contraintes	- Intégration dans une architecture existante sans rupture de service - Respect des règles d'habilitation (rôle administrateur ou service requis) - Données issues de plusieurs services à agréger en une seule réponse

Compétences associées (voir tableau)	- B2.1 – Concevoir et développer une solution applicative - B2.2 – Assurer la maintenance corrective ou évolutive d'une solution applicative - B2.3 – Gérer les données
---	---

Description des étapes de réalisation (démarche, méthodes, techniques utilisées)

1. ANALYSE DE L'EXISTANT

Avant de développer le nouvel endpoint, j'ai analysé la méthode `load()` déjà présente dans `BatchController`. Cette méthode retourne une vue complète et détaillée de l'état des batchs, en croisant les données de quatre services : `SourceService`, `BatchLiveService`, `BatchScheduleService` et `ConfService`. Elle sert de référence pour comprendre comment les données sont structurées et comment les services s'articulent entre eux.

J'ai identifié que pour un usage de supervision simplifié (listing des pipelines), seuls trois services étaient nécessaires : `SourceService` pour la liste des sources, `BatchLiveService` pour les statistiques en temps réel, et `ConfService` pour l'état de configuration.

2. DÉVELOPPEMENT DE L'ENDPOINT

J'ai créé la méthode `getAllPipelines()` dans `BatchController`, annotée `@GetMapping` avec le chemin `/domain/batch/{module}/getAllPipelines`.

La méthode commence par vérifier que l'utilisateur possède le rôle administrateur ou service via `getUser().checkRole()`, garantissant ainsi la sécurité de l'accès.

La logique d'agrégation parcourt ensuite les modules sources via `sourceService.getSource()`, puis pour chaque module, itère sur ses pipelines. Pour chacun, les informations suivantes sont collectées et ajoutées à la réponse :

- project : nom du module source (`s.getName()`)
- type : type du pipeline
- targets : cibles du pipeline (`StringUtils.join()`)
- throughput : débit calculé via `liveService.getStats().computeThroughput()`
- paused : état actif/pause récupéré depuis la configuration (`conf.getLive().paused()`)

Le résultat est retourné sous forme de `TypedMap` sérialisé en XML.

3. EXEMPLE DE RÉPONSE

```
<pipelines>
  <item>
    <paused>false</paused>
    <project>assurance</project>
    <throughput>0.0</throughput>
    <type>com.carrefour.fr.logm.apigee_x.assurance.MyPipeline</type>
    <targets>apigee-runtime</targets>
  </item>
</pipelines>
```

Conclusion	Cette mission m'a permis de consolider mes compétences en développement backend Java avec Spring Boot, en particulier sur la conception d'endpoints REST et l'agrégation de données multi-sources. L'analyse préalable de la méthode <code>load()</code> existante a été déterminante pour comprendre rapidement l'architecture et produire un résultat cohérent avec le système en place. Ce type d'endpoint léger répond à un besoin concret de supervision en exposant uniquement les données utiles au monitoring.
Productions associées	- Endpoint GET <code>getAllPipelines</code> intégré dans <code>BatchController</code> (dépôt LOGM, GitLab interne) - Réponse XML exploitable par les outils de supervision Grafana / dashboard APIM